



Pontificia Universidad  
**JAVERIANA**  
Cali

Facultad de Ingeniería y Ciencias  
Departamento de Electrónica y Ciencias de la Computación

# Carrera de Ingeniería Electrónica

## Sistemas Digitales

## Linux Processes

Febrero 2022

Tek



# Introduction

---

When you write a program for Linux (or UNIX and UNIX-like systems), you have to take into account that *the program will run in a multitasking environment*:

- multiple programs running at the same time
- sharing machine resources such as
  - memory,
  - disk space,
  - CPU cycles.

# Introduction

---

There may even be several instances of the same program running at the same time. It is important that these programs

- don't interfere with one another
- are aware of their environment
- and can act appropriately

to avoid conflicts such as trying to write the same file at the same time as another program.

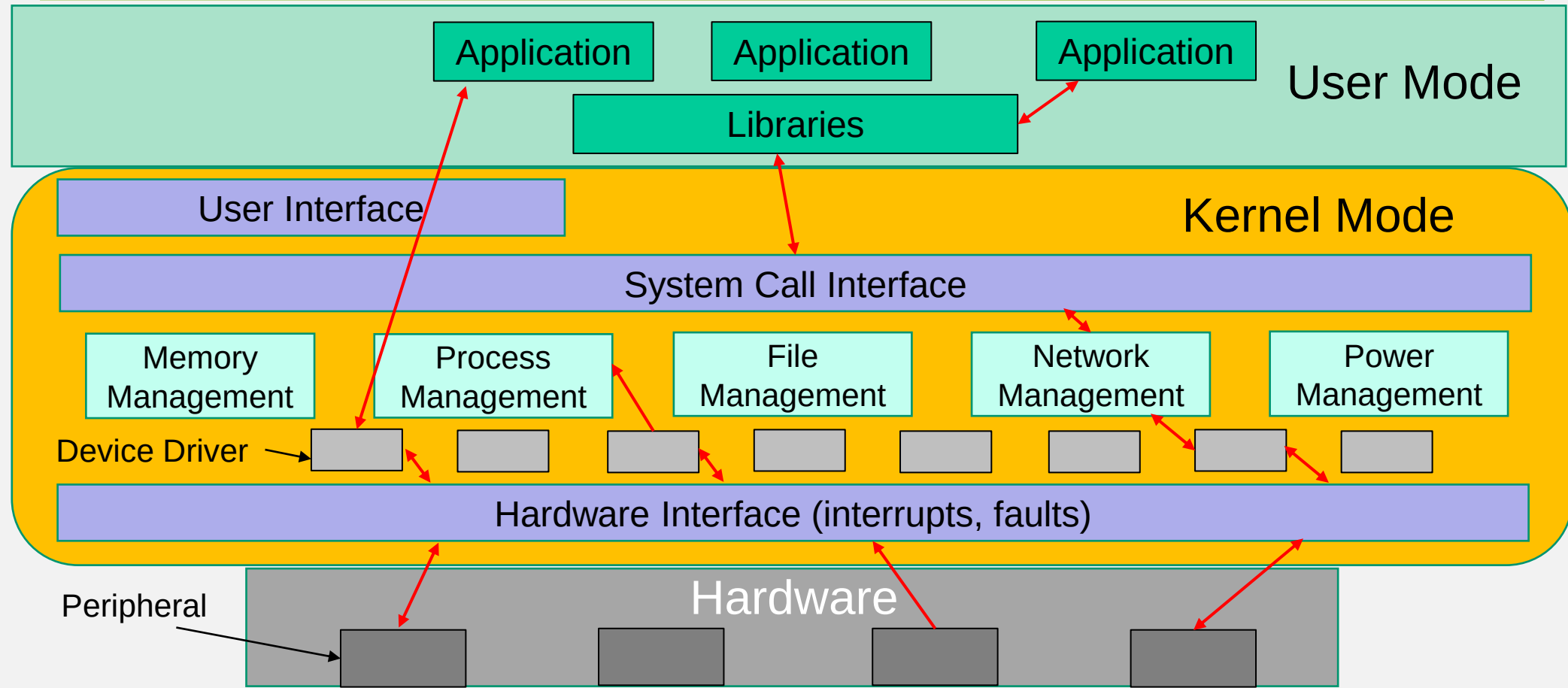
# What is an Operating System?

---

## Software

- *System Programs* - interface with the underlying computer hardware:
  - Device Drivers
  - Tool Chains: compilers, assemblers, linkers
  - *Operating System* – manages the physical resources of the computer:
    - » Shell, File Management
    - » Memory Hierarchies, I/O
    - » Interprocess communication and synchronisation
    - » Scheduler
    - » Dispatcher
- *Application Programs*

# Operating System



# What is a Kernel?

---

|             |                              |
|-------------|------------------------------|
| Kernel      | Interprocess Communication   |
|             | Interprocess Synchronisation |
| Microkernel | Process Scheduling           |
| Nanokernel  | Thread Management            |

Kernel Hierarchy  
(focus is on task management)

---

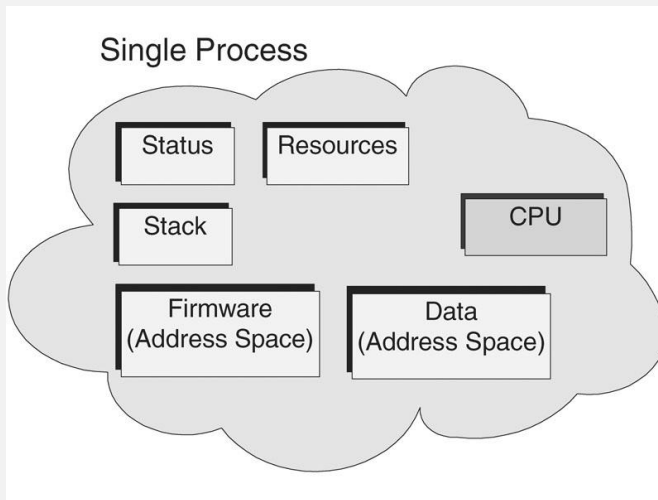
# What Is a Process?

---

- A program instance that is running (or *process*) is the logical unit of work scheduled by the operating system.
- A *process* consists of:
  - program code,
  - program data,
  - program stack,
  - open files (file descriptors),
  - an environment.
- The OS itself runs other programs to *manage system resources and control user access*.

# The CPU is a Resource

---

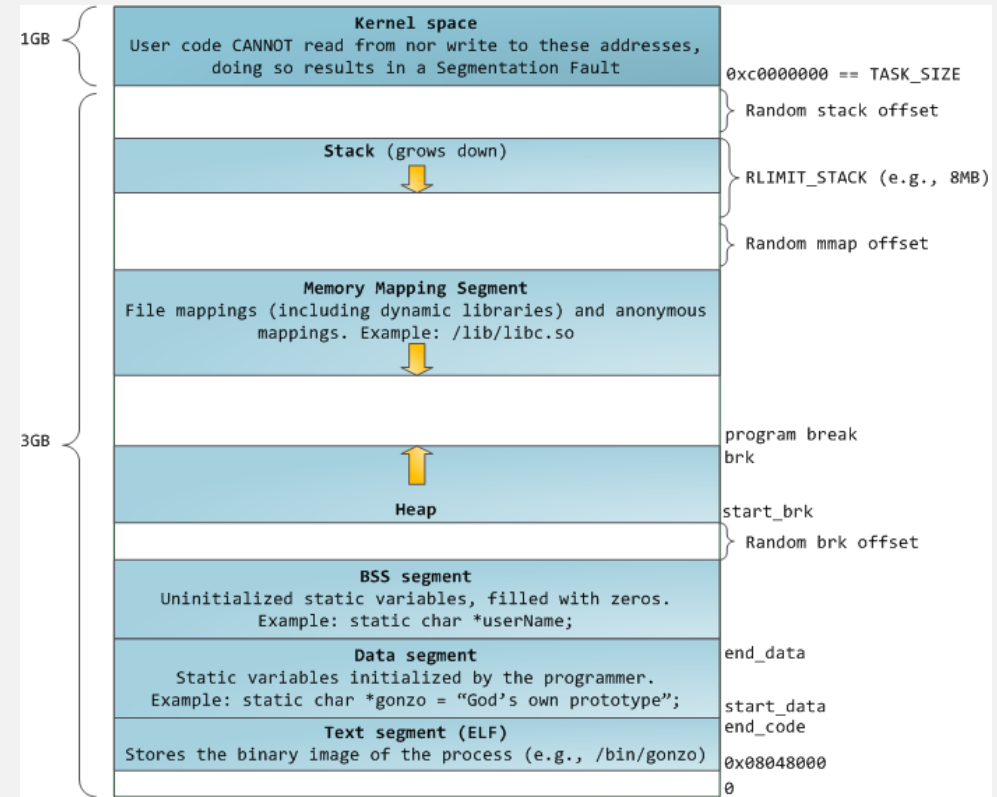


A process:

- Takes up space (memory) to store program code (perhaps in *firmware*), program data, program stack, status, and uses other systems resources.

# Memory is a Resource

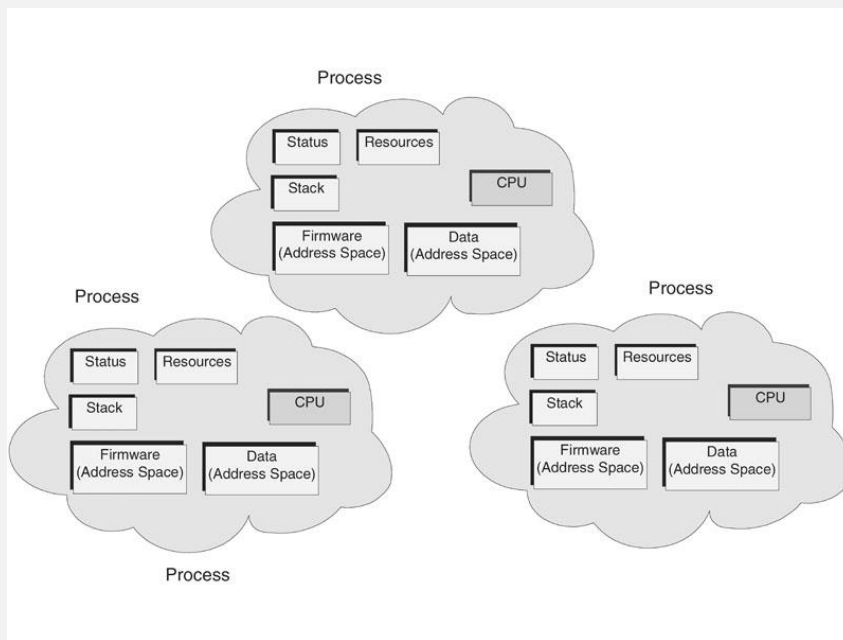
Each process in a multi-tasking OS runs in its own memory sandbox. This sandbox is the virtual address space, which on 32-bit x86 mode is always a 4GB block of memory addresses.



Standard segment layout in a Linux process

# The CPU is a Resource

What if there are multiple processes?

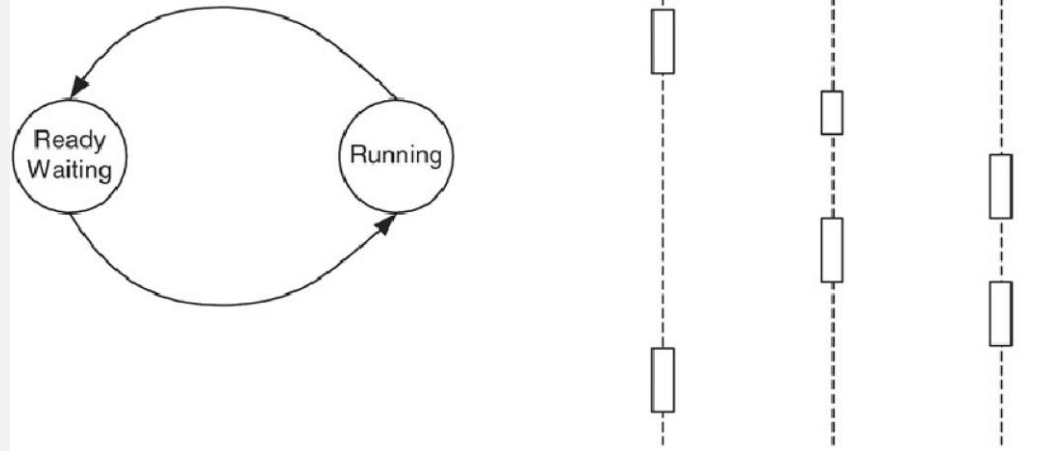


Manage

- How the CPU (and the resources, in general) is allocated to each process and for how long each process can retain the resources

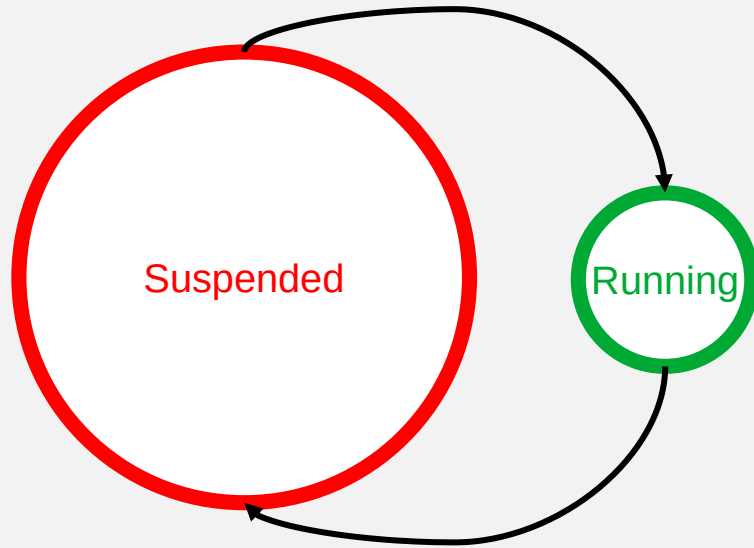
# Scheduling the CPU

The main resource, the CPU, is given to one process for a short while and then to other:

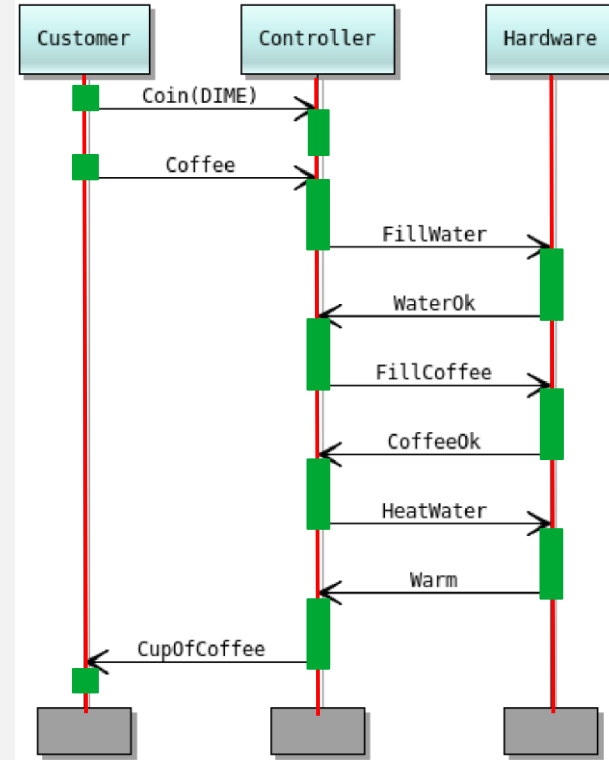


For a single-core CPU, only one process is actively executing; the CPU is thus *time-shared* at such a rate that it gives the *appearance of simultaneous execution*. Such a scheme is called *multitasking*; the processes are said to be running *concurrently*.

# Scheduling the CPU



Abstract view of states  
in an OS



MSC for the Coffee Machine

# What Is a Process?

---

- A *multitasking* operating system such as Linux lets many programs run at once.
- As a *multiuser* system, Linux allows many users to access the system at the same time. Each user can run many programs, or even many instances of the same program, at the same time.
- Programs often use *environment variables* to alter the way they work. Users can set the values of these environment variables.

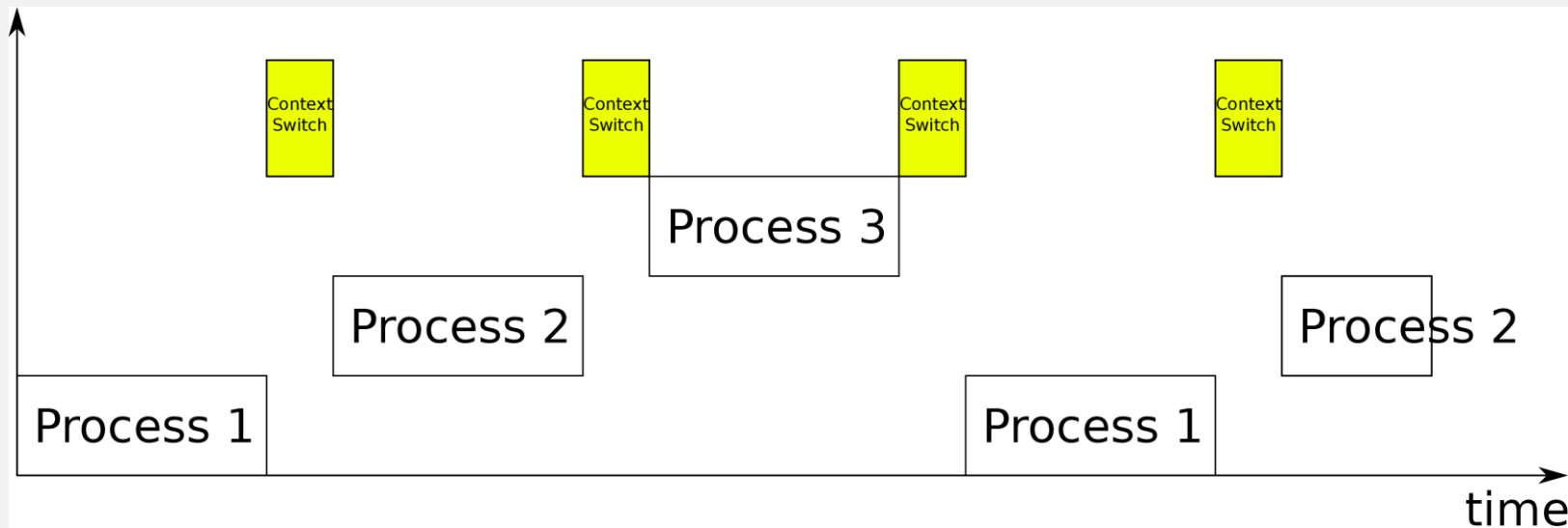
# How to control Processes?

---

- The Linux *Process Table* is like a data structure describing all of the processes that are currently loaded. It stores the process identifier (PId), a unique number for each currently executing program.
- In general, each process is started by another process known as its *parent* process (same as PARENT in SDL). A process so started is known as a *child* process (same as OFFSPRING in SDL).
- When Linux starts, it runs a single program, the prime ancestor and process number 1, *init*.

# Linux Process Scheduling

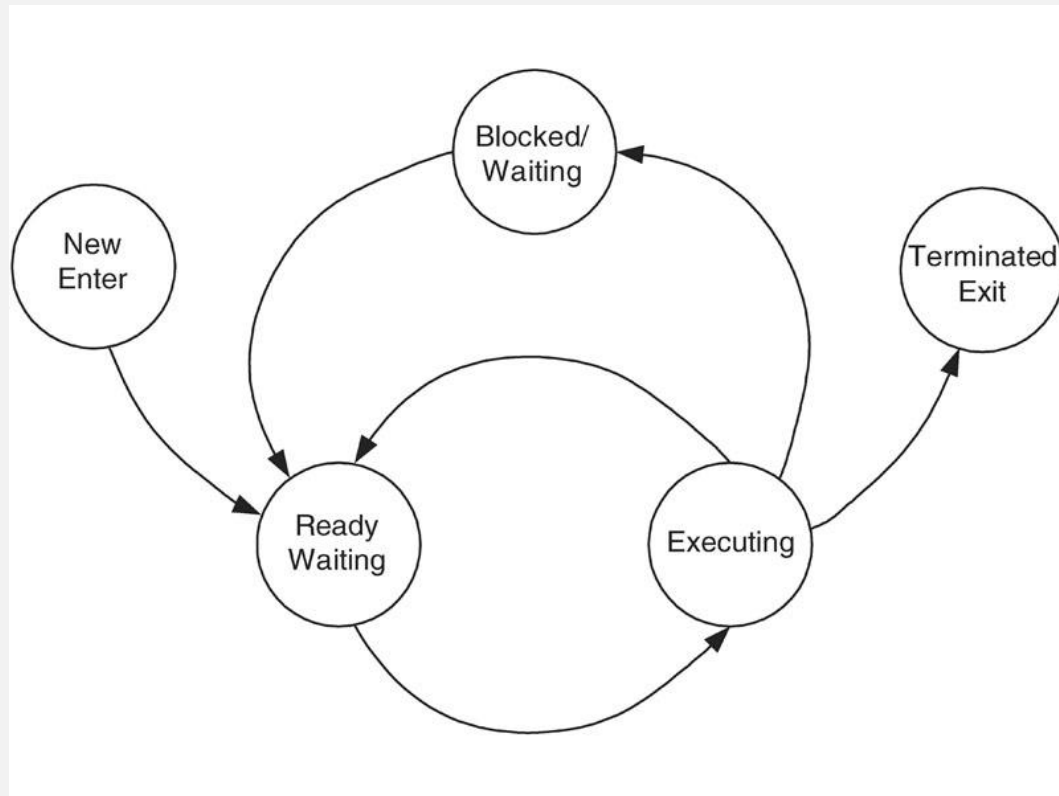
On a single-core computer, only one process can run at a time, while others wait their turn. These turns, known as *time slices*, are quite short thus giving the *impression* that programs are running at the same time.



# Switching Contexts

Context:

- Process state:
  - Program Counter
  - Values of variables
  - Register contents



Basic Diagram of Process States

# Linux Process Scheduling

---

- The Linux kernel uses a *process scheduler* to decide which process will receive the next time slice. It does this using the *process priority*:
  - Processes with a high priority get to run more often,
  - low-priority background tasks, run less frequently
- In a multitasking system such as Linux where several programs are likely to be competing for the same resource, programs that perform short bursts of work and pause for input are considered better behaved than those that hog the processor by continually calculating some value or continually querying the system to see if new input is available. Well-behaved programs are termed *nice* programs.

# Linux Process Scheduling

---

- Linux determines the priority of a process based on a “**nice**” value, which defaults to 0, and on the behaviour of the program.
  - Programs that run for long periods without pausing generally get lower priorities.
  - Programs that pause while, for example, waiting for input, get rewarded.
- This helps keep a program that interacts with the user responsive:
  - while it is waiting for some input from the user, the system increases its priority, so that when it's ready to resume, it has a high priority.

# Starting New Processes

---

- *system* – Executes a Program
    - runs the command passed to it as a string
    - waits for it to complete
  - *exec?* – Replaces a Process Image
  - *fork* – Duplicates a Process Image
    - duplicates the current process,
    - creates a new entry in the process table with many of the same attributes as the current process
    - the new process executes the **same code but with its own data space**, environment, and file descriptors
-

# Starting New Processes - fork

---

- Include files

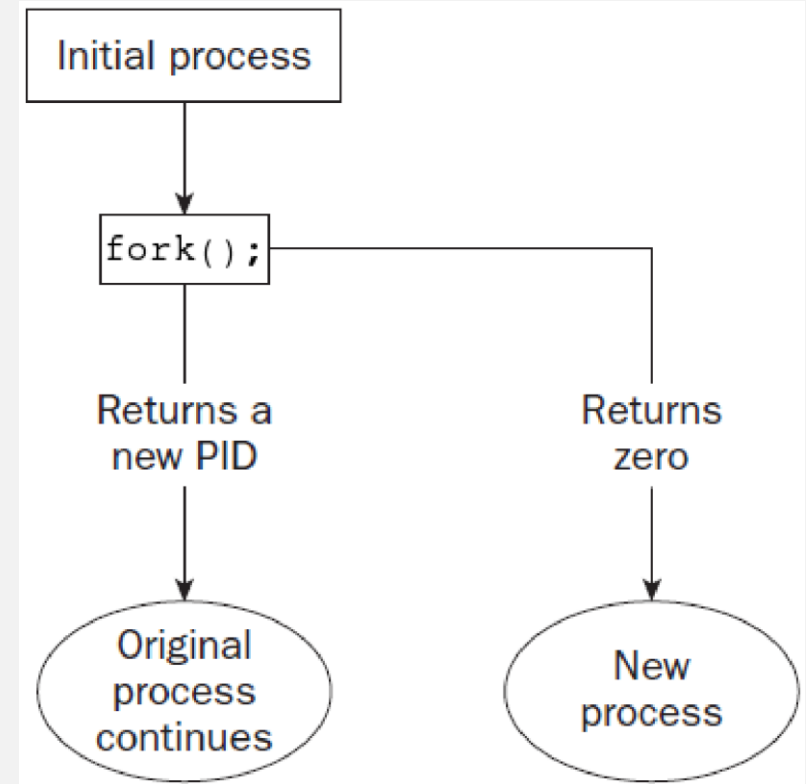
```
#include <sys/types.h>    /* recommended when using  
                           system calls, because  
                           some of their definitions  
                           use aliases for standard  
                           types that may change one  
                           day */  
  
#include <unistd.h>       /* provides access to the  
                           POSIX operating system  
                           API. */
```

- Function Prototype:

```
pid_t fork ( void );
```

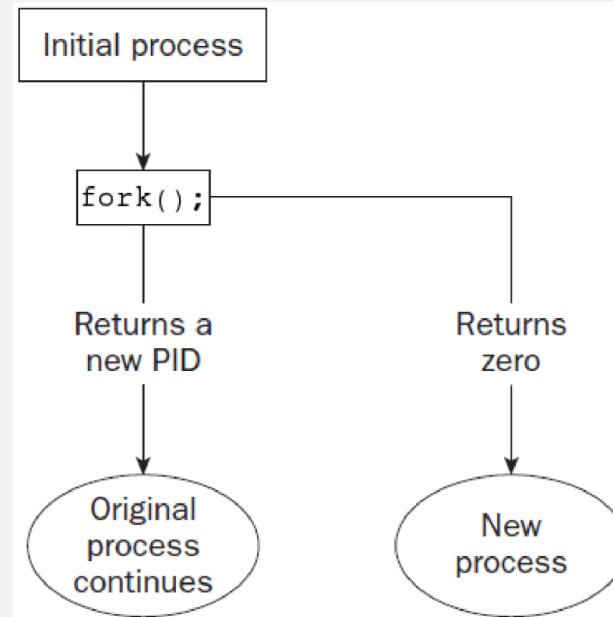
# Starting New Processes - `fork`

- The call to `fork` in the parent returns the PID of the new child process.
- The new process continues to execute just like the original, with the exception that in the child process the call to `fork` returns 0.
- This allows both the parent and child to determine which is which.



# Starting New Processes - fork

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main ( void )
7 {
8     pid_t pid;
9     char *message;
10    int    n;
11
12    printf ( "fork program starting\n" );
13    pid = fork ();
14    switch ( pid )
15    {
16        case -1:
17            perror ( "fork failed" );
18            exit ( 1 );
19        case 0:
20            message = "This is the child";
21            n = 5;
22            break;
23        default:
24            message = "This is the parent";
25            n = 3;
26    }
27
28    for ( ; n > 0; n-- )
29    {
30        puts ( message );
31        sleep ( 1 );
32    }
33    exit ( 0 );
34 }
```



# Waiting for a Process - `wait`

---

- `wait` – Waits for a process
  - causes a parent process to pause until one of its child processes is stopped.
  - returns the PId of the child process.

# Waiting for a Process - wait

---

- Include files

```
#include <sys/types.h>    /* recommended when using
                           system calls, because
                           some of their definitions
                           use aliases for standard
                           types that may change one
                           day */

#include <sys/wait.h>      /* macros related to
                           wait */
```

- Function Prototype:

```
pid_t wait ( int *stat_loc );
```

# Waiting for a Process - wait

---

```
        :

pid = fork ();
switch ( pid )
{
    case -1:

        :

        default:
            message = "This is the parent";
            n = 3;
    }

for ( ; n > 0; n-- )

    :

/* This section of the program waits for the child process to finish. */
1 if ( pid )
2 {
3     int  stat_val;
4     pid_t child_pid;
5
6     child_pid = wait ( &stat_val );
7
8     printf ( "Child has finished: PID = %d\n", child_pid );
9     if ( WIFEXITED ( stat_val ) )
10         printf ( "Child exited with code %d\n", WEXITSTATUS ( stat_val ) );
11     else
12         printf ( "Child terminated abnormally\n" );
13 }
```

# Processes - Summary

---

- Linux processes can cooperate, can send each other messages, and can interrupt one another.
- They can even arrange to share segments of memory between themselves, but they are essentially separate entities within the operating system.
- They do not readily share variables.

# Further Reading

---

MATTHEW, Neil; STONES, Rick. **Beginning Linux Programming, 4th Edition**. Wiley Publishing, 2008. **Chapters 11, 12.**

PECKOL, James K. **Embedded Systems – A Contemporary Design Tool**. John Wiley & Sons, Inc., 2008. **Chapter 11.**

LAPLANTE, Phillip. **Real-Time Systems Design and Analysis, Third Edition**. John Wiley & Sons, Inc. 2004. **Chapter 6.**